

Speaker Notes

Predicting Marathon Race Performance Using Machine Learning

IECE 465 · Final Project · Spring 2026

Sanad Sahawneh · Brian Allen · Mahib Rahman

SLIDE 1 • Title (~15 sec · Sanad)

Good morning, everyone. We're Sanad, Brian, and Mahib, and our final project is **Predicting Marathon Race Performance Using Machine Learning**. Over the next ten minutes we'll walk you through what we built, what we learned, and we'll close with a quick live demo.

SLIDE 2 • Agenda (~25 sec · Sanad)

Quick roadmap. We'll start with where we left off at Progress Update Two, then talk about what's new in the final version — there's a lot. After that: the pipeline, our evaluation framework, the leaderboard, and the practical-accuracy results. Then the more reflective part — lessons learned and the obstacles we had to solve technically. We'll wrap with individual contributions, a snapshot of our project website, and a live demo prediction.

Let me hand it over to Mahib to recap.

SLIDE 3 • Recap: Where We Left Off (~50 sec · Mahib)

Thanks Sanad. Quick recap of where we were a couple of weeks ago.

Our dataset is the 2023 Boston Marathon — about **26,598 runners**. Three input features: age group, gender, and half-marathon split. The target is the final net finish time in seconds.

At Update Two we had three models running: a baseline ratio predictor, linear regression, and a neural network. The pipeline went load, clean, split, train, evaluate, visualize.

The numbers on the right are where we landed. Best RMSE was the linear regression at **9.70 minutes**, neural net just behind at 9.75, baseline at 10.11. R-squared around 0.95 across the board.

The honest limitation: we had a working pipeline, but no ensemble model, no outlier filtering, the best model wasn't auto-selected, and nothing was being saved as a deliverable. So that was the gap we set out to close.

SLIDE 4 • What's New in the Final Version (~60 sec · Brian)

Six concrete improvements since Update Two.

First, we added a **bagged-trees ensemble** — that's our fourth model, compared head-to-head against the others.

Second, **data cleaning version two**. We removed invalid times and unrealistic finish-to-half ratios, anything outside 1.80 to 3.20, which catches data-entry errors and runners who clearly didn't finish.

Third, **auto best-model selection**. The system now ranks models by RMSE and picks the winner programmatically — no manual decision.

Fourth, **practical accuracy metrics**. Median absolute error and the percent-within-X-minutes bands. These answer the "is this actually useful?" question, which raw RMSE doesn't.

Fifth, every output now gets saved — results table, predictions, threshold table, figures, and a .mat package of the best model. Everything is reproducible.

And sixth, the quick demo prediction we'll run at the end.

The takeaway is at the bottom: we went from a working pipeline to a reproducible, auto-evaluated ML system.

Sanad will take you through the pipeline.

SLIDE 5 · Final ML Pipeline (~50 sec · Sanad)

Thanks Brian. Eight stages, fully reproducible — we set the random seed to 42 so anyone re-running this gets the same split and the same numbers.

Load, clean, split, encode, train, evaluate, select, export. The flow is intentionally linear so each stage can be debugged in isolation.

Two callouts at the bottom worth pausing on. The clean stage on the left does three things: drops missing rows, requires positive race times, and applies the finish-to-half ratio filter. That last one is the most useful — a marathon finish is normally **just over twice** a half-marathon split, so anything below 1.8 or above 3.2 is almost certainly not a real finishing pattern.

On the right, the four candidate models. Baseline, linear regression, the neural network from Update Two, and the new bagged-trees ensemble — 150 learners. Each one gets the same train and test split, the same feature encoding, the same evaluation. Apples to apples.

Mahib will walk through how we evaluate them.

SLIDE 6 • Expanded Evaluation Framework (~50 sec · Mahib)

Thanks Sanad. We went from four metrics to nine.

On the left, what we kept from Update Two — MAE, RMSE, MAPE, and R-squared. Each one answers a different question, so we kept all four. MAE for average error in human units, RMSE when we want to penalize outliers, MAPE for relative scale, R-squared for goodness of fit.

What's new is on the right. **Median absolute error** is robust to outliers in a way the regular MAE isn't. And then the three within-time bands — five, ten, and fifteen minutes. That's how runners actually think about predictions: not "what's the RMSE", but "is this prediction within ten minutes of my real time?" We also wired all of these into the auto-ranking, so the leaderboard you'll see next is sorted on real metrics, not eyeballed.

Sanad will take you through the leaderboard.

SLIDE 7 • Final Results: Leaderboard (~60 sec · Sanad)

Thanks Mahib. Here are all four models ranked by RMSE on the held-out test set.

The winner — auto-selected by the script — is the **bagged-trees ensemble**. RMSE of **9.42 minutes**, MAE of 6.31, R-squared 0.953, and **78% of test runners predicted within ten minutes** of their actual finish.

But here's what's interesting — and this is one of our lessons learned: the linear regression came in at 9.70 RMSE, only about three-tenths of a minute behind. The neural network was 9.75. The baseline was 10.11.

So the ensemble wins, but barely. The honest takeaway is that the half-marathon split feature is doing most of the work, and once you have that signal, complicated models barely beat simple ones.

One quick note — these numbers are from the run we report in the final submission. If you re-run the script you'll get something very close but not identical, because of the random split.

SLIDE 8 • Practical Accuracy & Decisions (~50 sec · Brian)

Two charts here that translate the model into something useful.

On the left, the within-time accuracy bands for the best model. **52% of predictions** are within five minutes — that's the "excellent" band. **78% within ten minutes**. And **91% within fifteen**. So nine

out of ten runners get a prediction that's at worst fifteen minutes off, which for a four-hour marathon is meaningful.

On the right, the over/under decision extension. We picked five benchmark times — three hours, three-thirty, four, four-thirty, five — and asked: did the model correctly predict whether the runner would finish under that line? Accuracy ranges from **94% to 98%**, depending on the threshold. The harder calls are around 3:30 — that's where the bulk of the runners are, so the line splits the distribution most evenly. Easy benchmarks like 5:00 are nearly all "under" anyway.

SLIDE 9 • Lessons Learned *(~50 sec · Sanad)*

Four big takeaways from the project itself.

One — **one feature dominates**. The half-marathon split alone explains most of the variance. Age and gender helped, but barely.

Two — **simple beats complex when the signal is linear**. Like I just said, the ensemble only edged out the linear regression by 0.3 minutes RMSE. The linear model is far easier to explain and would be the better deployment choice in most contexts.

Three — **units matter for interpretation**. "RMSE equals 583 seconds" is technically correct but useless to a runner. Switching everything to minutes, plus the within-X-minute bands, made our results actually mean something to a non-technical reader.

Four — **reproducibility is a feature**. Saving every output — tables, figures, the model package — turned a one-off script into something a future student or our team six months from now could re-run cleanly. That mattered more than we expected.

SLIDE 10 • Obstacles & Technical Solutions *(~60 sec · Brian)*

Four real bugs we had to solve.

First — **categorical encoding mismatch**. When we split into train and test, sometimes a rare age group would appear in test but not train, and the one-hot encoder would silently break. **Fix:** we define the category lists from the full dataset before splitting, then force both halves to use them. No surprises.

Second — **invalid race times**. Raw CSV had zero splits, missing fields, and impossible finish-to-half ratios from data-entry errors. **Fix:** the cleaning stage Brian mentioned earlier — rmmissing, positivity check, and the ratio filter.

Third — **feature scale broke the neural net**. half_time_sec is in the thousands; the one-hot features are zero or one. The network wouldn't converge. **Fix:** standardize using mean and standard deviation computed only on training data, then reuse those parameters on test. No data leakage, stable training.

Fourth — and this is the interpretation one — **errors in seconds were unreadable**. "RMSE = 583 sec" doesn't communicate anything. **Fix:** every metric reports in minutes, plus the within-X-minute bands.

Sanad will take us through team contributions.

SLIDE 11 • Individual Contributions (~35 sec total • Everyone)

[Sanad — ~12 sec]

Quick rundown of who owned what. I led the ML system integration — the end-to-end pipeline, the evaluation framework expansion, the auto best-model selection, the visualizations, and orchestrating the demo we're about to show.

[Brian — ~12 sec]

I built the original code skeleton, owned the data preprocessing and the cleaning version two work — including designing the ratio-filter bounds — and handled performance interpretation and presentation prep.

[Mahib — ~12 sec]

I worked on feature encoding and normalization, model tuning — both the neural net architecture and the ensemble parameters — performance validation across runs, and the threshold-extension validation you saw on slide eight.

SLIDE 12 • Project Website (~30 sec • Sanad)

One last thing before the demo — we built the framework page of our project website that pulls everything together. Our plan is to have the site have the full project overview, dataset description, the model leaderboard you saw, an **interactive** prediction widget where you can pick age group, gender, and half-split and see a live prediction, all our visualizations, and our team page.

SLIDE 13 • Live Demonstration (~50 sec · Sanad)

Quick live demo. We're going to predict a finish time for a sample runner using the saved best model from the .mat file we exported.

The runner: age group 40 to 44, male, half-marathon split of one hour and forty-two minutes — so 102 minutes.

The script does three things. **One** — loads the saved best-model package. **Two** — encodes the runner using the same category lists we trained on, so no encoding mismatch. **Three** — runs the prediction.

Predicted finish: **3 hours, 36 minutes, 42 seconds**. Actual finish for that runner was 3:34:18.

Absolute error: **2 minutes and 24 seconds**. That puts this prediction in the "excellent" band — under five minutes off.

One sample isn't a result, of course — the leaderboard numbers are the real story. But it shows the pipeline works end-to-end, from saved model to single-runner prediction.

SLIDE 14 • Thank You / Questions (~15 sec · Sanad)

Three takeaways: **half-marathon split is the dominant signal, practical metrics beat raw error, and a reproducible pipeline** turned this into a real system.

Thanks for listening!